PX4 + ROS 2 Drone Control

Hands-on RISEx Workshop

Author:
Dion Walton

Applied Nonlinear Control Lab

Faculty of Electrical & Computer Engineering

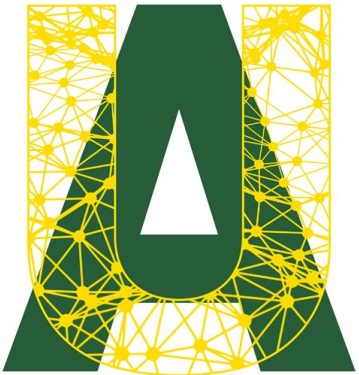
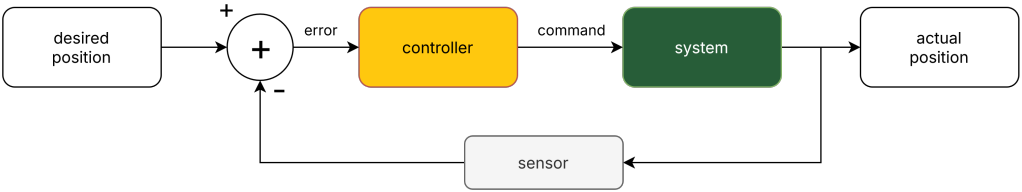


Table of Contents

1. **Basics of Control**
2. **Drone Architecture**
3. **PX4 Autopilot Concepts**
4. **ROS2 Concepts**
5. **PX4 $\leftrightarrow$ ROS2 Communication**
6. **Simulation**
7. **Safety Features**
8. **Workshop Tasks** (What you will do)

How Do You Control a Drone?

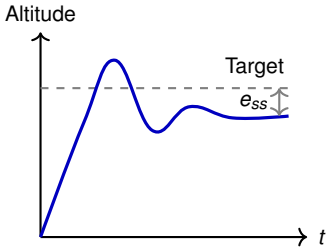
- **Problem**: Suppose you want to make a program that can control where a drone is in **3D space**.
- **Solution**: Use sensors to approximate the **current** location vs. the **desired** and generate force that brings the drone in that direction



Step 1 — Proportional (P): The Present

$$u(t) = K_p e(t)$$

- **Proportional (P)**: Drives the drone using the **present** error.
  - Scales thrust directly with current distance from target altitude.
- Generates a strong initial climb when far from the setpoint.
- Establishes the core restoring force that pulls the drone toward the target.

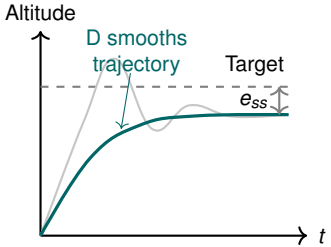


P: Fast climb toward the target.

Step 2 — Adding Derivative (D): The Future

$$u(t) = K_p e(t) + K_d \frac{\mathrm{d}e(t)}{\mathrm{d}t}$$

- **Derivative (D)**: Shapes the approach using the **future** trend.
  - Reacts to climb rate ($\frac{\mathrm{d}e}{\mathrm{d}t}$), gently braking thrust as the drone nears target altitude.
- Smooths rapid motion and damps oscillations from *P*.
- Delivers a clean, stable arrival without overshooting.

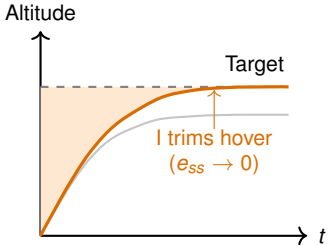


P + D: Smooth, damped approach.

Step 3 — Adding Integral (I): The Past

$$u(t) = K_p e(t) + K_d \frac{\mathrm{d}e(t)}{\mathrm{d}t} + K_i \int_0^t e(\tau)\mathrm{d}\tau$$

- **Integral (I)**: Locks onto the target using **past** error history.
  - Accumulates any remaining offset over time to trim thrust upward.
- Supplies the steady hover thrust needed to balance gravity and wind.
- Holds the drone precisely at the target altitude ($e_{ss} \to 0$).



P + D + I: Precise target hold.

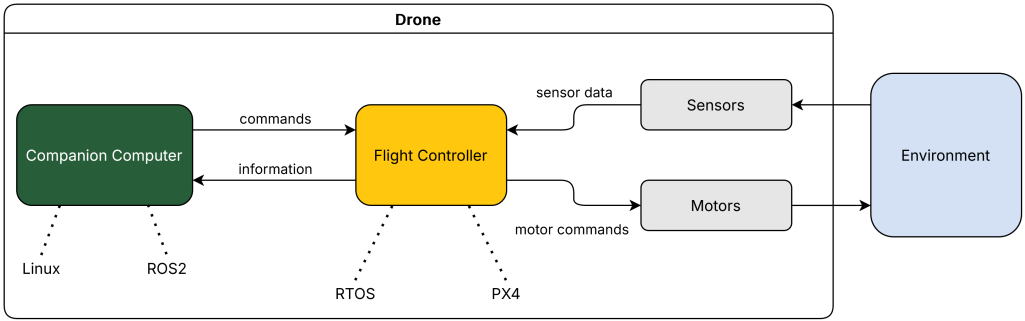
What's in a Drone?

- **Two** computers make up a typical autonomous drone.

1. **Flight Controller (PX4)**
   - Interfaces directly with sensors & motors.
   - Runs a Real-Time Operating System (**RTOS**).
   - Does deterministic, millisecond-level control math (PID).

2. **Companion Computer (ROS 2)**
   - Runs **Ubuntu**.
   - Handles heavy computations, visual tasks, and ROS 2 communication.

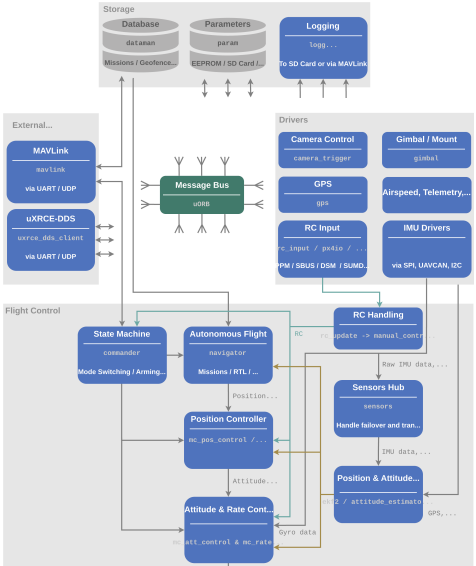
Drone Architecture



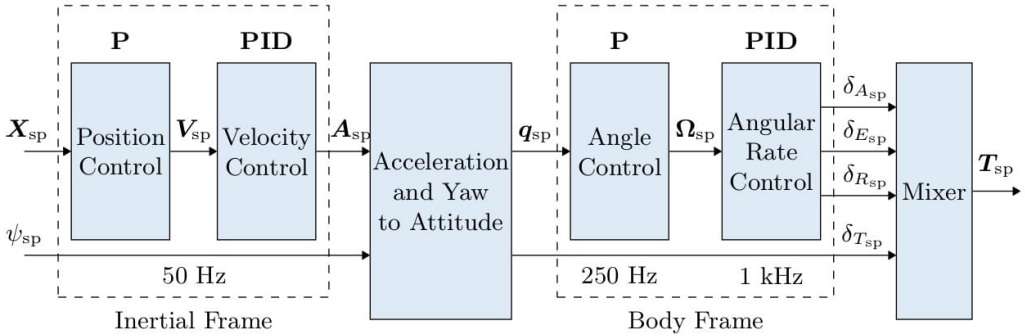
What is PX4 Autopilot?

- **PX4 Autopilot**: Open-source flight stack handling estimation and control for autonomous vehicles.
- Uses a cascaded controller architecture to break control into distinct stages.
- Fuses raw sensor data (GPS, IMU, Magnetometer) in the Extended Kalman Filter (**EKF2**).
- Features a light, lock-free, asynchronous distributed internal messaging system called **uORB**.
- Manages **flight modes** (e.g., Acro, Manual, Position, Offboard).

PX4's Architecture



PX4's Multirotor Control Pipeline



Source: PX4 Controller Diagrams

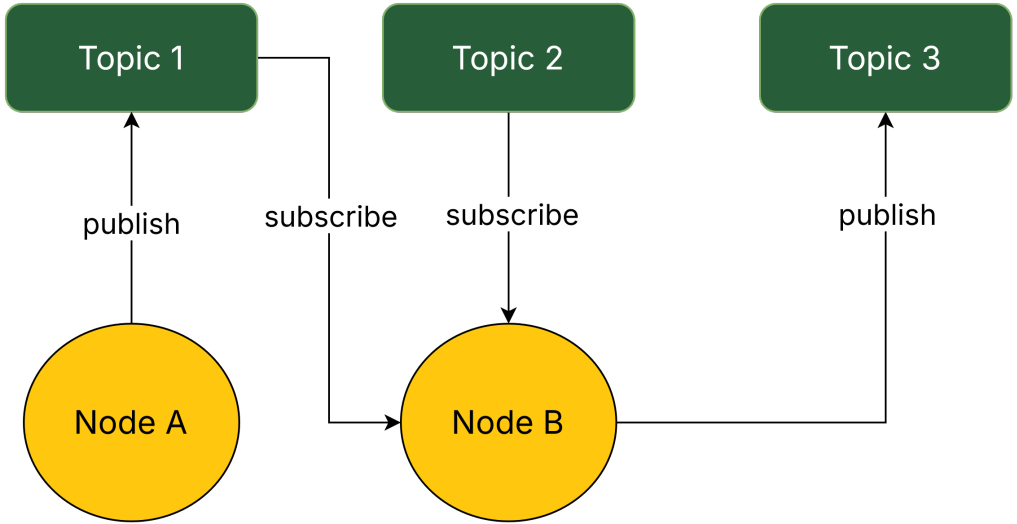
What is ROS 2?

- **ROS 2**: **R**obot **O**perating **S**ystem **2**: Framework and library suite for building complex robotic software.

! *Not an OS itself*. Runs on top of Ubuntu Linux.

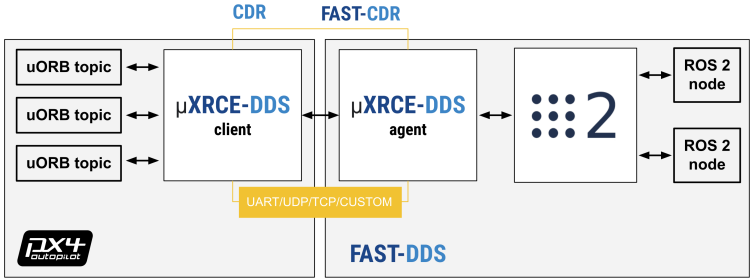
- **Node**: An individual process responsible for a specific task.
- **Topic**: A data channel that nodes publish to or subscribe from.
  - **Publisher**: **Sends** structured messages to a topic.
  - **Subscriber**: **Receives** messages from a topic.

ROS 2: Nodes & Topics



PX4 & ROS 2 Communication

- **Micro XRCE-DDS**: Client-server middleware bridging resource-constrained microcontrollers with DDS networks.
- **Micro XRCE Agent**: Broker bridging ROS 2 DDS. Runs on the **companion computer**.
- **Micro XRCE Client**: Lightweight client sending uORB requests. Runs on the **flight controller**.



px4_msgs & Time Synchronization

- px4_msgs **Package**: ROS 2 definitions mirrored directly from PX4 uORB messages.
  - Examples: `VehicleLocalPosition.msg`, `TrajectorySetpoint.msg`, `VehicleCommand.msg`.
- **Timestamp Alignment**:
  - Every message requires a `timestamp` field in **microseconds** (μs).
  - In C++: `msg.timestamp = this->get_clock()->now().nanoseconds() / 1000;`
- **Quality of Service (QoS)**: PX4 DDS topics use **Best Effort** reliability to minimize telemetry latency.

PX4 Offboard Mode Protocol

- **Offboard Mode**: Relinquishes local autopilot decision-making to external setpoints published via ROS 2.
- **The Heartbeat Requirement**:
  - PX4 requires continuous setpoints streamed at > 2 **Hz** *before* and *during* Offboard mode.
  - If streaming drops for > 0.5 s, PX4 triggers an automatic failsafe (e.g., Hold mode or Land).
- **Primary Bridge Topics**:
  - `/fmu/in/offboard_control_mode`: Enables position, velocity, or acceleration loops.
  - `/fmu/in/trajectory_setpoint`: Sends target position (x, y, z), velocity, or acceleration vectors.
  - `/fmu/in/vehicle_command`: Sends Arm/Disarm and Flight Mode transition commands.

Safety Systems & Operational Procedures

- **Hardware Requirements**:
  - Power on the physical RC transmitter **before** arming.
  - Configure an emergency **Kill Switch** on an RC toggle switch to override ROS 2 immediately.
- **PX4 Autonomous Failsafes**:
  - **Offboard Signal Loss**: Automatically switches to Position Hold or Return-to-Launch (RTL).
  - **Geofence**: Hard radius and altitude limits enforced by PX4.
  - **Battery Failsafe**: Auto-lands when supply voltage drops below minimum cell thresholds.

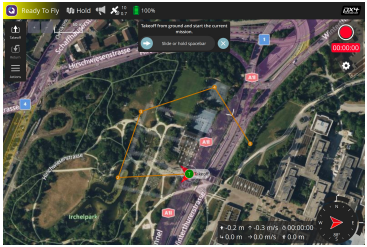
Simulation Stack: Software-In-The-Loop (SITL)

- **Why SITL?** Safely test autonomous flight algorithms before hardware deployment.
- **Complete Software Architecture**:

Gazebo Simulator $\longleftrightarrow$ **PX4 SITL Stack** $\longleftrightarrow$ **Micro XRCE Agent** $\longleftrightarrow$ **ROS 2 Node**

- **Real-Time Hardware Emulation**: Runs exact PX4 flight software binary compiled for Linux.
- **Physics Environment**: Gazebo models quadrotor dynamics, gravity, aerodynamic drag, and IMU/GPS sensors.

Simulation Tools: QGroundControl & Gazebo


Ready To Fly
Hold
100%
Takeoff from ground and start the current mission
Slide or hold spacebar
00:00:00
Universitätsstraße
Universitätsstraße
Universitätsstraße

ROS 2 Offboard Node Control Loop

- **Standard Execution Sequence for Autonomous Takeoff**:

1. **Initialize**: Create ROS 2 publishers for `offboard_control_mode` and `trajectory_setpoint`.
2. **Pre-Arm Stream**: Spin a timer callback (100 Hz) publishing dummy setpoints for at least 10–20 cycles.
3. **Arm Vehicle**: **Arm** the motors.
4. **Position Switch**: Go PX4's **position mode** via QGC/Radio switch.
5. **Offboard Switch**: Go to PX4's **offboard mode** via QGC/Radio switch.
6. **Setpoint Loop**: Stream desired target acceleration/position setpoints.

Debugging & Telemetry Inspection

- **Verify DDS Connection**:
  `$ ros2 topic list | grep fmu`
- **Inspect Drone Position Output**:
  `$ ros2 topic echo /fmu/out/vehicle_local_position`
- **Inspect Vehicle Arming State**:
  `$ ros2 topic echo /fmu/out/vehicle_status`
- **PX4 Shell Debugging**:
  `px4> listener vehicle_local_position`

ROS 2 Workspace Structure

- **Source Directory Layout**:

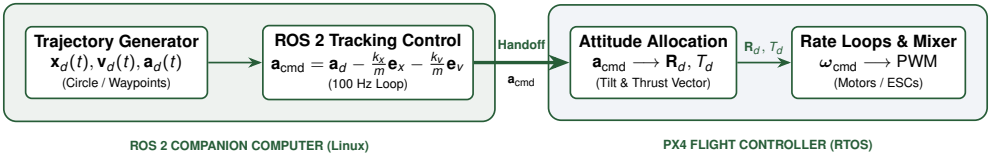
```
px4-dev-toolkit-workshop/
            ros2/src/workshop/src/workshop_offboard_node.cpp
```

- **Build Workflow**:

```
$ cd px4-dev-toolkit-workshop/
$ ./ros2/build.sh sitl
$ source ros2/runtime_env.sh
```

High-Level Architecture: Trajectory & Control Handoff

- **Companion Computer (ROS 2)**: Generates target trajectories ($\mathbf{x}_d, \mathbf{v}_d, \mathbf{a}_d$) and calculates desired acceleration vectors $\mathbf{a}_{\text{cmd}}$ at 100 Hz.
- **Control Handoff (Micro XRCE-DDS)**: Bypasses PX4's position/velocity PIDs, streaming $\mathbf{a}_{\text{cmd}}$ via `/fmu/in/trajectory_setpoint`.
- **Flight Controller (PX4)**: Converts 3D acceleration into target tilt angles (Attitude) and thrust, running fast inner-loop rate controllers (> 250 Hz).



Workshop Node: Setup & Parameters

- **ROS 2 Parameters**: Dynamically configurable options for flight testing:
  - Mass: `mass` (2.0 kg)
  - Gains: `kx` (2.0), `kv` (2.0)
  - Trajectory Selector: `trajectory_mode` (`"circle"`, `"figure_eight"`, `"waypoints"`)
- **High-Rate Execution Loop**:
  - Main timer callback runs at 100 **Hz (**10 **ms)**.
  - Guarantees setpoint streaming well above PX4's 2 Hz offboard heartbeat threshold.

Tasks 1 & 2: Odometry Subscription

- **Task 1: Subscribe to EKF2 Odometry Topic**

```
// Subscribe to high-frequency state estimation from EKF2
vehicle_odometry_sub_ = create_subscription<px4_msgs::msg::
    VehicleOdometry>(
    "/fmu/out/vehicle_odometry",
    qos,
    std::bind(&WorkshopOffboardNode::odometry_callback, this, std::
    placeholders::_1)
);
```

- **Task 2: Extract State Vectors in Callback**

```
current_pos_ = Eigen::Vector3d(msg->position[0], msg->position[1], msg->
    position[2]);
current_vel_ = Eigen::Vector3d(msg->velocity[0], msg->velocity[1], msg->
    velocity[2]);
```

Task 3: Waypoint Trajectory Generation

- **Objective**: Implement a piecewise time-based state machine to sequence 4 spatial setpoints.
- **Safety Bounding Box Constraints**:
  - Horizontal Boundary: $|x| \leq 1.0\,\text{m}, \quad |y| \leq 1.0\,\text{m}$
  - Altitude Boundary: $-1.2\,\text{m} < z < -0.6\,\text{m}$ (*NED frame, z points down*)
- **Task 3 State Logic (10-Second Dwell Time per Waypoint)**:

```
if (t < 10.0)       pos_d = Vector3d( 0.5,  0.5, -0.8); // WP 1
else if (t < 20.0)  pos_d = Vector3d( 0.5, -0.5, -0.8); // WP 2
else if (t < 30.0)  pos_d = Vector3d(-0.5, -0.5, -0.8); // WP 3
else                pos_d = Vector3d(-0.5,  0.5, -0.8); // WP 4
```

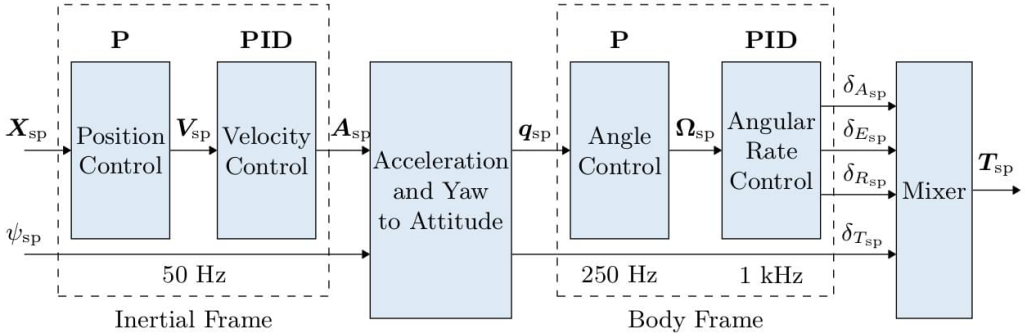
Task 4: Control Law Acceleration Tracking

- **Parametric Trajectories**: Node computes desired position $\mathbf{x}_d(t)$, velocity $\mathbf{v}_d(t)$, and feedforward acceleration $\mathbf{a}_d(t)$.
- **Task 4 (Feedforward-PD Acceleration Command)**:

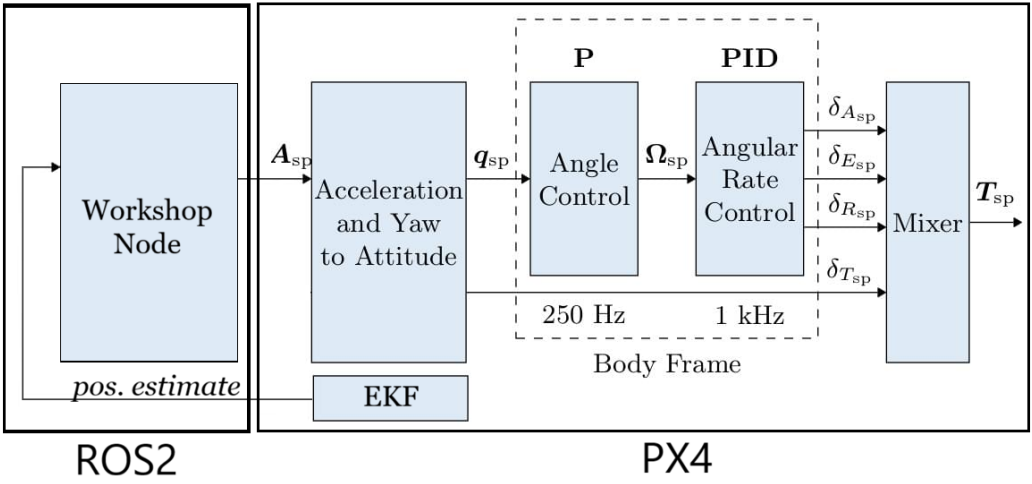
$$\mathbf{a}_{\text{cmd}} = \mathbf{a}_d - \frac{k_x}{m}(\mathbf{x} - \mathbf{x}_d) - \frac{k_v}{m}(\mathbf{v} - \mathbf{v}_d)$$

- **PX4 Offboard Acceleration Payload**:
  - `OffboardControlMode`: Set `acceleration = true`, disable position/velocity flags.
  - `TrajectorySetpoint`: Stream $\mathbf{a}_{\text{cmd}}$ while setting `position` and `velocity` vectors to `NAN`.

Recall: PX4's Multirotor Control Pipeline



Workshop Node Position/Velocity Bypass



Terminal Workflow Cheatsheet

1. **Terminal 1 (Start SITL, QGC, and Micro XRCE-DDS Agent)**:
   ```
   $ cd px4-dev-toolkit-workshop/ && ./tools/sitl start
   ```

2. **Terminal 2 (Build & Run ROS 2 Offboard Control Node)**:
   ```
   $ ./ros2/build.sh sitl
   $ source ./ros2/runtime_env.sh
   $ ros2 run workshop_offboard_control workshop_offboard_node
     -ros-args -p trajectory_mode:="waypoints"
   ```

References I

- PX4 Autopilot User Guide & Multirotor Controller Architecture,
  PX4 Development Team.
  https://docs.px4.io
- ROS 2 Documentation (Jazzy Jalisco Release),
  Open Source Robotics Foundation (OSRF).
  https://docs.ros.org/en/jazzy/
- eProsma Micro XRCE-DDS Client & Agent Middleware,
  eProsma.
  github.com/eProsma/Micro-XRCE-DDS
- Eigen v3 C++ Template Library for Linear Algebra,
  G. Guennebaud, B. Jacob, et al.
  https://eigen.tuxfamily.org

References II



Gazebo Physics Simulator & QGroundControl GCS,
Open Robotics & Dronecode Foundation.
gazebosim.org | qgroundcontrol.com

Questions & Discussion

Thank You!

PX4 + ROS 2 Drone Control Workshop

Get the Workshop Code & Slides

Repository:
`github.com/ANCL/px4-dev-toolkit-workshop`

Authors: Dion Walton

Lab: Applied Nonlinear Control Lab (ANCL), UofA